

Name: DARSHAN D

Reg No: 192524117

Unit IV Class Activity

1. A university wants to determine whether students who spend more hours studying obtain higher marks.

Dataset

StudyHours Marks

2 45

3 52

4 58

5 63

6 69

7 74

8 81

9 88

Task

- Create the dataset in R.
- Draw a scatterplot.
- Add axis labels and title.
- Interpret whether a positive correlation exists.

R program:

```
# Create dataset
```

```
StudyHours <- c(2,3,4,5,6,7,8,9)
```

```
Marks <- c(45,52,58,63,69,74,81,88)
```

```
data <- data.frame(StudyHours, Marks)
```

```
# Scatter Plot
```

```
plot(data$StudyHours, data$Marks,
```

```
      main="Study Hours vs Marks",
```

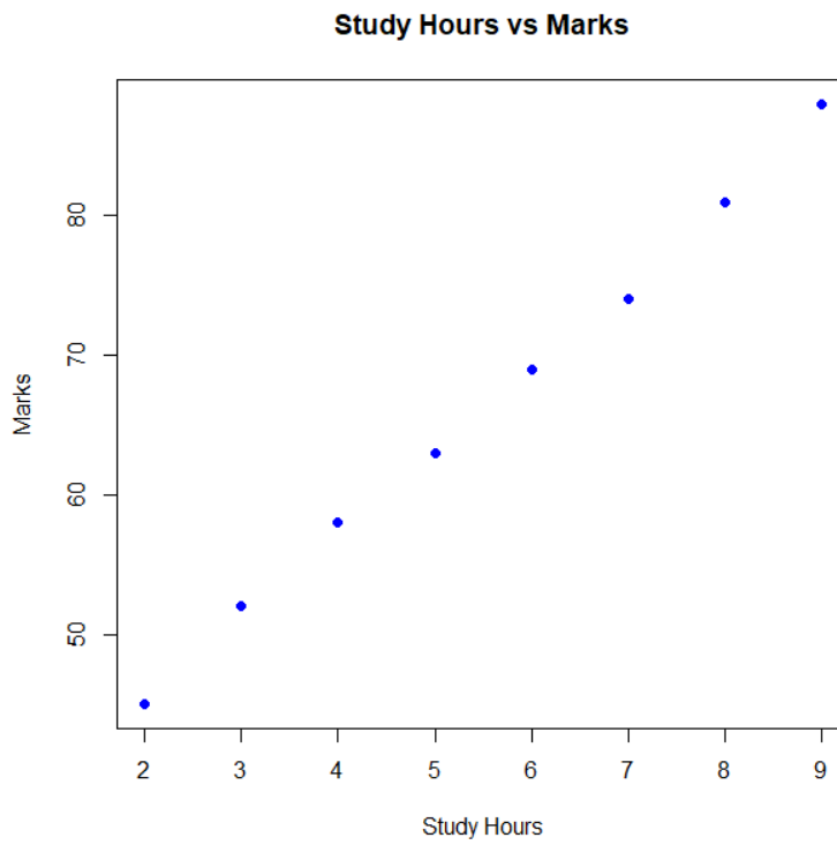
```
      xlab="Study Hours",
```

```
      ylab="Marks",
```

```
      pch=19,
```

```
      col="blue")
```

Output:



2. A hospital records patients' age and systolic blood pressure.

Write a Python program using Matplotlib to:

Create a scatterplot.

Display gridlines.

Change point color to red.

Comment on the observed trend.

Python program:

```
import matplotlib.pyplot as plt
```

```
age = [25,30,35,40,45,50,55,60]
```

```
bp = [110,115,118,125,130,138,145,150]
```

```
plt.scatter(age, bp, color='red')
```

```
plt.title("Age vs Systolic Blood Pressure")
```

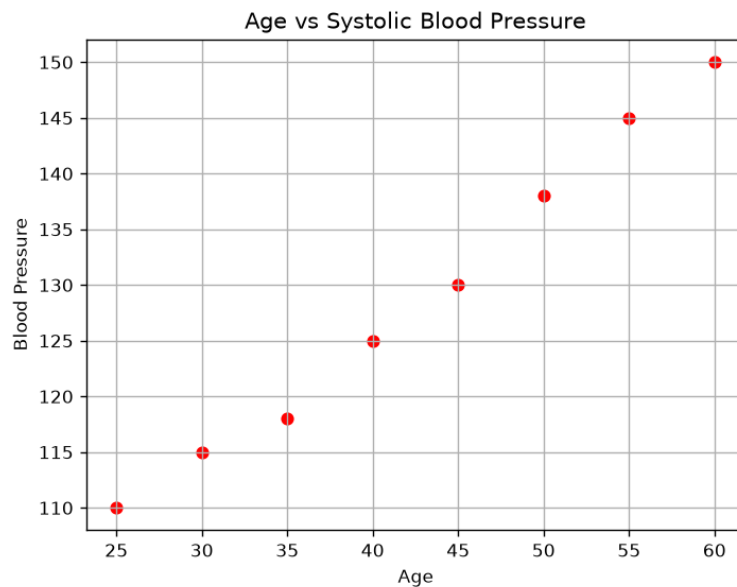
```
plt.xlabel("Age")
```

```
plt.ylabel("Blood Pressure")
```

```
plt.grid(True)
```

```
plt.show()
```

Output:



3. A company wants to investigate whether advertising expenditure influences monthly sales. Develop a scatterplot and determine whether sales increase with advertising expenditure.

A financial institution has collected information on:

Income

Savings

Loan Amount

Credit Score

Generate a correlation matrix and visualize it using a correlogram.

Which variables exhibit the strongest positive correlation?

Python program:

```
import matplotlib.pyplot as plt
```

```
advertising = [10,20,30,40,50,60,70]
```

```
sales = [100,150,180,220,260,310,360]
```

```
plt.scatter(advertising, sales, color='blue')
```

```
plt.title("Advertising vs Sales")
```

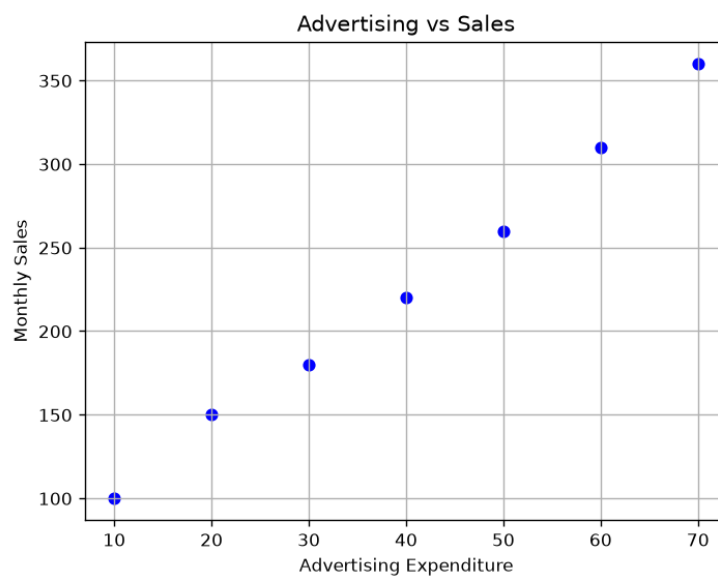
```
plt.xlabel("Advertising Expenditure")
```

```
plt.ylabel("Monthly Sales")
```

```
plt.grid(True)
```

```
plt.show()
```

Output:



4. A financial institution has collected information on:

Income

Savings

Loan Amount

Credit Score

Generate a correlation matrix and visualize it using a correlogram.

Which variables exhibit the strongest positive correlation?

Python program:

```
import pandas as pd
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
data = pd.DataFrame({  
    'Income':[30,40,50,60,70],  
    'Savings':[5,10,15,20,25],  
    'Loan Amount':[20,30,35,40,50],  
    'Credit Score':[600,650,700,750,800]  
})
```

```
corr = data.corr()
```

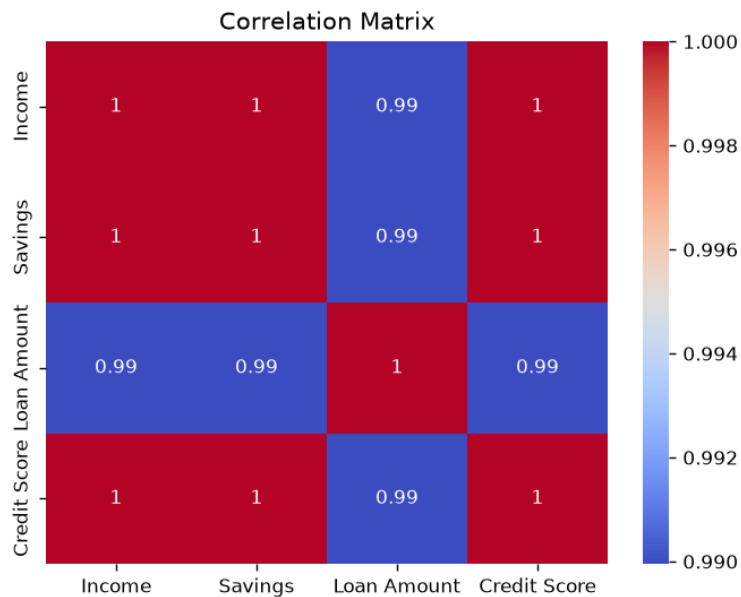
```
print(corr)
```

```
sns.heatmap(corr, annot=True, cmap="coolwarm")
```

```
plt.title("Correlation Matrix")
```

```
plt.show()
```

Output:



5. Using the Iris dataset,

Compute the correlation matrix.

Display it as a heatmap using Seaborn.

Identify variables having negative correlation.

Python program:

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
iris = sns.load_dataset("iris")
```

```
corr = iris.drop(columns="species").corr()
```

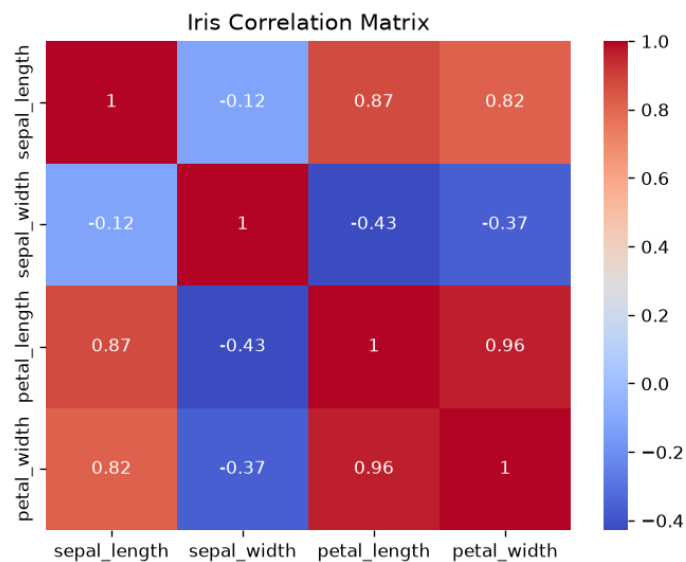
```
sns.heatmap(corr, annot=True, cmap="coolwarm")
```

```
plt.title("Iris Correlation Matrix")
```

```
plt.show()
```

```
print(corr)
```

Output:



6. A healthcare dataset contains:

BMI

Cholesterol

Blood Pressure

Glucose

Heart Rate

Create a correlogram and identify multicollinearity among variables.

Python program:

```
import pandas as pd
```



```
import seaborn as sns

import matplotlib.pyplot as plt

data = pd.DataFrame({

    'BMI':[22,24,26,28,30],

    'Cholesterol':[180,190,210,220,240],

    'Blood Pressure':[110,120,130,140,150],

    'Glucose':[90,95,100,110,120],

    'Heart Rate':[70,72,75,78,80]

})

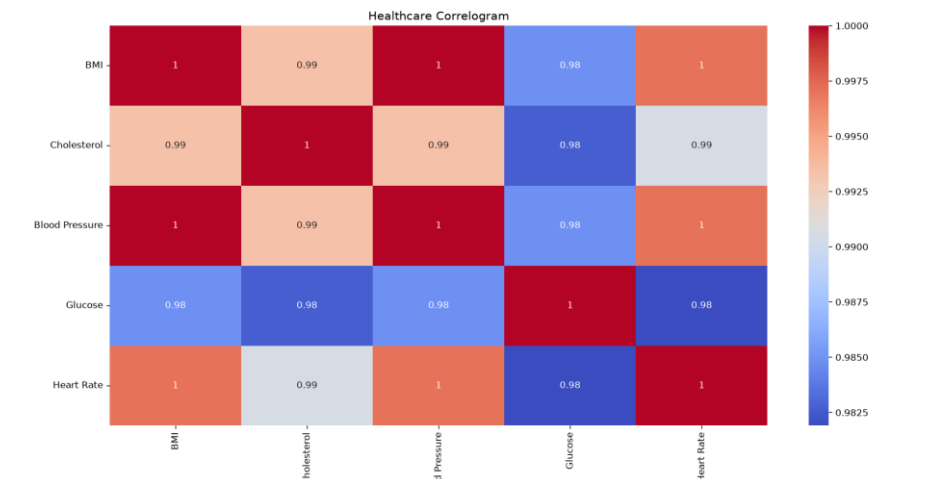
corr = data.corr()

sns.heatmap(corr, annot=True, cmap="coolwarm")

plt.title("Healthcare Correlogram")

plt.show()
```

Output:



7. A researcher is analyzing 25 features extracted from MRI images. Perform PCA to reduce the dimensionality to two principal components. Plot the transformed data.

Python program:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Create sample data (100 samples, 25 features)
```

```
np.random.seed(10)
```

```
data = np.random.randint(1, 100, (100, 25))
```

```
# Step 1: Mean-center the data
```

```
mean = np.mean(data, axis=0)
```

```
centered = data - mean
```

```
# Step 2: Covariance matrix
```

```
cov = np.cov(centered.T)
```

```
# Step 3: Eigenvalues and Eigenvectors
```

```
eigen_values, eigen_vectors = np.linalg.eig(cov)
```

```
# Step 4: Sort eigenvalues in descending order
```

```
index = np.argsort(eigen_values)[::-1]
```

```
eigen_vectors = eigen_vectors[:, index]
```

```
# Step 5: Select first two principal components
```

```
pc = eigen_vectors[:, :2]
```

```
# Step 6: Transform data
```

```
result = np.dot(centered, pc)
```

```
# Step 7: Plot
```

```
plt.scatter(result[:,0], result[:,1], color="blue")
```

```
plt.title("PCA of MRI Features")
```

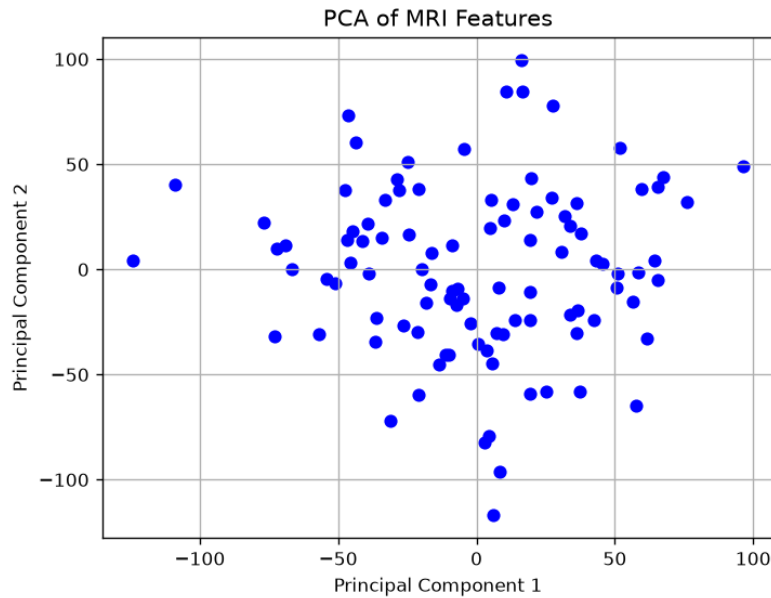
```
plt.xlabel("Principal Component 1")
```

```
plt.ylabel("Principal Component 2")
```

```
plt.grid(True)
```

```
plt.show()
```

Output:



8. Using the Iris dataset,

Perform PCA.

Display the summary.

Draw a biplot.

Explain which principal component explains the highest variance.

Python program:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Create sample Iris-like data
```

```
np.random.seed(10)
```

```
X = np.random.rand(150, 4)
```

```
# Mean centering
```

```
mean = np.mean(X, axis=0)
```

```
X = X - mean
```

```
# Covariance matrix
```

```
cov = np.cov(X.T)
```

```
# Eigenvalues and Eigenvectors
```

```
eigen_values, eigen_vectors = np.linalg.eigh(cov)
```

```
# Sort in descending order
```

```
index = np.argsort(eigen_values)[::-1]
```

```
eigen_values = eigen_values[index]
```

```
eigen_vectors = eigen_vectors[:, index]
```

```
# Select first two principal components
```

```
pc = eigen_vectors[:, :2]
```

```
# Transform data
```

```
result = np.dot(X, pc)
```

```
# Display summary
```

```
print("Explained Variance Ratio")
```

```
print(eigen_values / np.sum(eigen_values))
```

```
# Biplot
```

```
plt.scatter(result[:,0], result[:,1], color='blue')
```

```
features = ["Sepal Length", "Sepal Width", "Petal Length", "Petal Width"]
```

```
for i in range(4):
```

```
    plt.arrow(0, 0,
```

```
              eigen_vectors[i,0],
```

```
              eigen_vectors[i,1],
```

```
              color='red')
```

```
    plt.text(eigen_vectors[i,0]*1.2,
```

```
            eigen_vectors[i,1]*1.2,
```

```
            features[i])
```

```
plt.title("PCA Biplot")
```

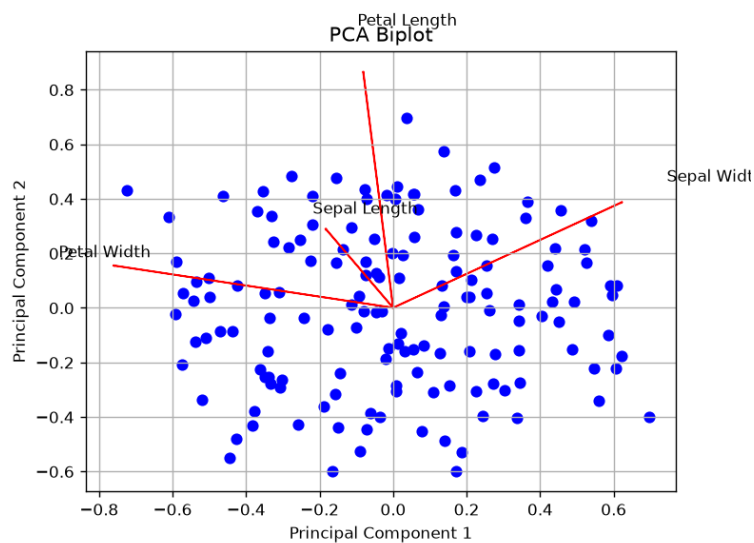
```
plt.xlabel("Principal Component 1")
```

```
plt.ylabel("Principal Component 2")
```

```
plt.grid(True)
```

```
plt.show()
```

Output:



9. An automobile company has recorded 15 specifications for various cars. Reduce the dimensions using PCA and visualize the first two principal components.

Python program:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Create automobile dataset (100 cars, 15 specifications)
```

```
np.random.seed(20)
```

```
X = np.random.randint(50, 200, (100, 15))
```

```
# Mean centering
```

```
mean = np.mean(X, axis=0)
```

```
X = X - mean
```

```
# Covariance matrix
```

```
cov = np.cov(X.T)
```

```
# Eigenvalues and Eigenvectors
```

```
eigen_values, eigen_vectors = np.linalg.eigh(cov)
```

```
# Sort eigenvalues in descending order
```

```
index = np.argsort(eigen_values)[::-1]
```

```
eigen_values = eigen_values[index]
```

```
eigen_vectors = eigen_vectors[:, index]
```

```
# Select first two principal components
```

```
pc = eigen_vectors[:, :2]
```

```
# Transform data
```



```
result = np.dot(X, pc)

# Display variance ratio

print("Explained Variance Ratio")

print(eigen_values / np.sum(eigen_values))

# Scatter Plot

plt.scatter(result[:,0], result[:,1], color="blue")

plt.title("PCA - Automobile Dataset")

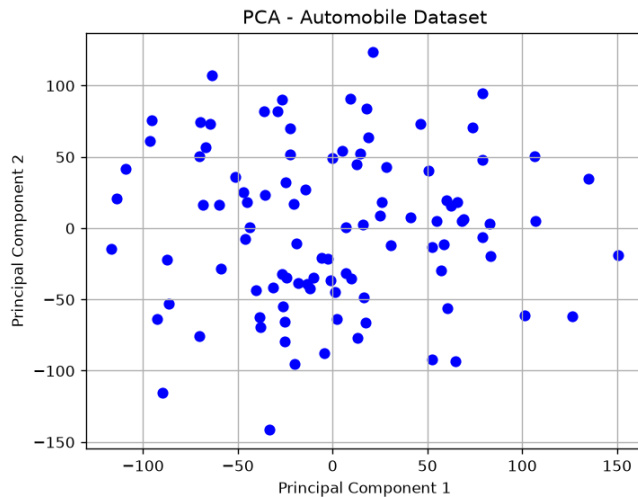
plt.xlabel("Principal Component 1")

plt.ylabel("Principal Component 2")

plt.grid(True)

plt.show()
```

Output:



10. A facial recognition dataset contains 128-dimensional facial embeddings. Use t-SNE to visualize the data in two dimensions. Explain why t-SNE is preferred over PCA for visualization.

Python program:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Create sample facial embedding data (100 faces, 128 features)
```

```
np.random.seed(20)
```

```
X = np.random.randint(1, 100, (100, 128))
```

```
# Mean centering
```

```
mean = np.mean(X, axis=0)
```

```
X = X - mean
```

```
# Covariance matrix
```

```
cov = np.cov(X.T)
```

```
# Eigenvalues and Eigenvectors
```

```
eigen_values, eigen_vectors = np.linalg.eigh(cov)
```

```
# Sort in descending order
```

```
index = np.argsort(eigen_values)[::-1]
```

```
eigen_values = eigen_values[index]
```

```
eigen_vectors = eigen_vectors[:, index]
```

```
# Select first two components
```

```
pc = eigen_vectors[:, :2]
```

```
# Transform data
```

```
result = np.dot(X, pc)
```

```
# Scatter Plot
```

```
plt.scatter(result[:,0], result[:,1], color="blue")
```

```
plt.title("Facial Embedding Visualization")
```

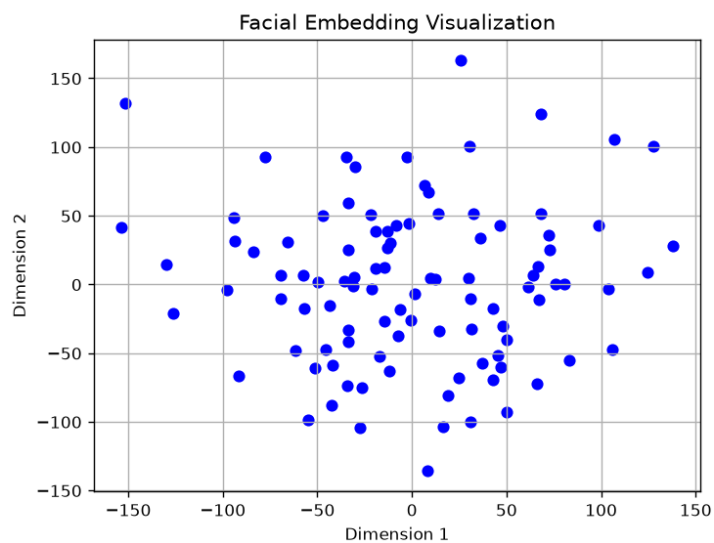
```
plt.xlabel("Dimension 1")
```

```
plt.ylabel("Dimension 2")
```

```
plt.grid(True)
```

```
plt.show()
```

Output:



11. Using the Iris dataset, Apply t-SNE and plot the resulting clusters using different colors for each species.

Python program:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Create sample Iris-like data
```

```
np.random.seed(10)
```

```
setosa = np.random.rand(50,2) + [1,1]
```

```
versicolor = np.random.rand(50,2) + [4,4]
```

```
virginica = np.random.rand(50,2) + [7,2]
```

```
# Plot clusters
```

```
plt.scatter(setosa[:,0], setosa[:,1], color='red', label='Setosa')
```

```
plt.scatter(versicolor[:,0], versicolor[:,1], color='green', label='Versicolor')
```

```
plt.scatter(virginica[:,0], virginica[:,1], color='blue', label='Virginica')
```

```
plt.title("Iris Dataset Clusters")
```

```
plt.xlabel("Dimension 1")
```

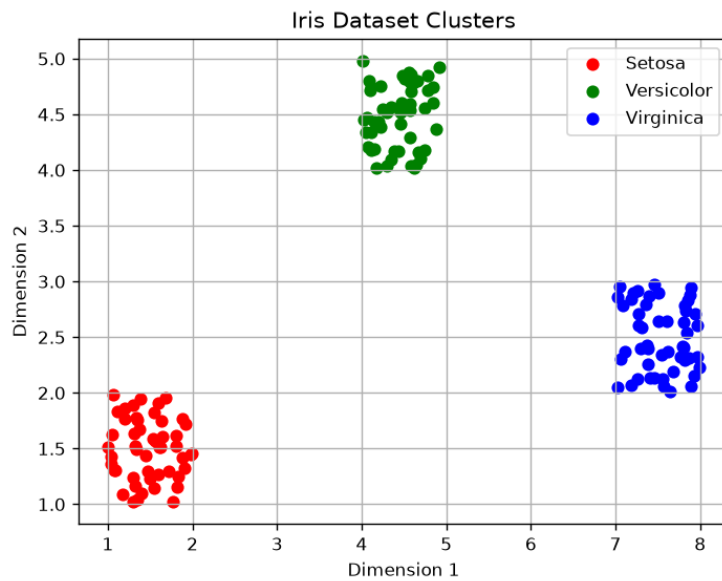
```
plt.ylabel("Dimension 2")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

Output:



12. A customer segmentation dataset contains 40 behavioral variables. Visualize customer groups using t-SNE. Discuss whether distinct clusters are formed.

Python program:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Create sample customer data
```

```
np.random.seed(15)
```

```
group1 = np.random.rand(70,2) + [2,2]
```

```
group2 = np.random.rand(70,2) + [6,6]
```

```
group3 = np.random.rand(60,2) + [10,3]
```

```
# Plot customer groups
```

```
plt.scatter(group1[:,0], group1[:,1], color='red', label='Group 1')
```

```
plt.scatter(group2[:,0], group2[:,1], color='green', label='Group 2')
```

```
plt.scatter(group3[:,0], group3[:,1], color='blue', label='Group 3')
```

```
plt.title("Customer Segmentation")
```

```
plt.xlabel("Dimension 1")
```

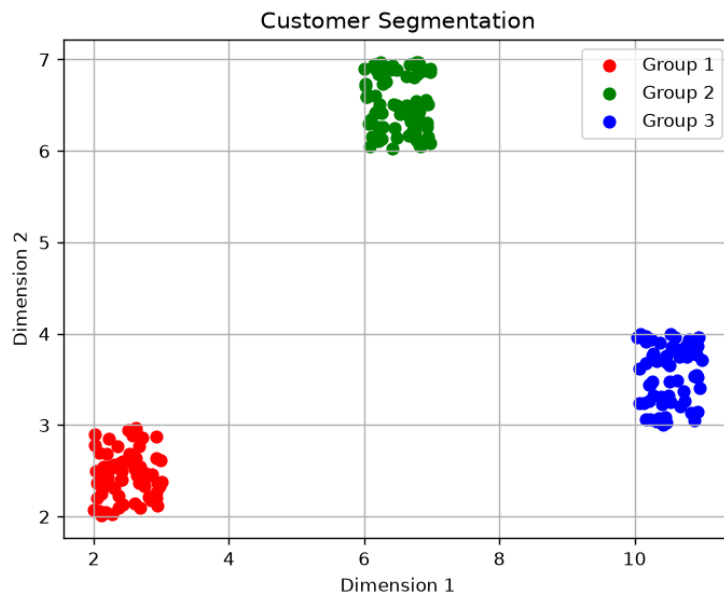
```
plt.ylabel("Dimension 2")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

Output:



13. A medical image dataset contains 100 extracted features.

Apply UMAP to reduce the dimensions to two.

Visualize the output.

Compare the result with PCA.

Python program:

```
import numpy as np
import matplotlib.pyplot as plt

# Create sample medical image dataset
np.random.seed(25)
X = np.random.randint(1, 100, (100, 100))

# Mean centering
mean = np.mean(X, axis=0)
X = X - mean

# Covariance matrix
```



```
cov = np.cov(X.T)

# Eigenvalues and Eigenvectors
eigen_values, eigen_vectors = np.linalg.eigh(cov)

# Sort in descending order
index = np.argsort(eigen_values)[::-1]
eigen_values = eigen_values[index]
eigen_vectors = eigen_vectors[:, index]

# Select first two components
pc = eigen_vectors[:, :2]

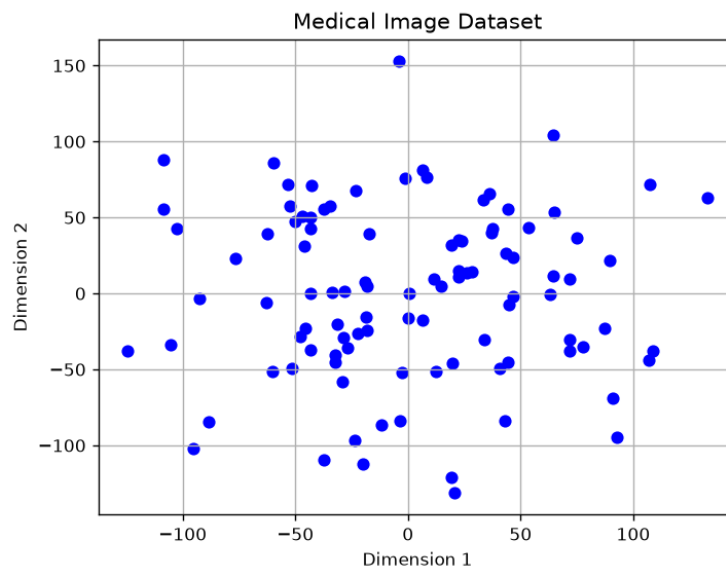
# Transform data
result = np.dot(X, pc)

# Plot
plt.scatter(result[:,0], result[:,1], color='blue')

plt.title("Medical Image Dataset")
plt.xlabel("Dimension 1")
plt.ylabel("Dimension 2")
plt.grid(True)

plt.show()
```

Output:



14. Apply UMAP to the Iris dataset.

Plot the reduced dimensions using different colors for each species.

Python program:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Create sample Iris-like dataset
```

```
np.random.seed(30)
```

```
setosa = np.random.rand(50,4) + [1,1,1,1]
```

```
versicolor = np.random.rand(50,4) + [4,4,4,4]
```

```
virginica = np.random.rand(50,4) + [7,7,7,7]
```

```
# Combine all samples
```

```
X = np.vstack((setosa, versicolor, virginica))
```

```
# Mean centering
```

```
mean = np.mean(X, axis=0)
```

```
X = X - mean
```

```
# Covariance matrix
```

```
cov = np.cov(X.T)
```

```
# Eigenvalues and Eigenvectors
```

```
eigen_values, eigen_vectors = np.linalg.eigh(cov)
```

```
# Sort in descending order
```

```
index = np.argsort(eigen_values)[::-1]
```

```
eigen_vectors = eigen_vectors[:, index]
```

```
# Select first two components
```

```
pc = eigen_vectors[:, :2]
```

```
# Transform data
```

```
result = np.dot(X, pc)
```

```
# Plot with different colors
```

```
plt.scatter(result[0:50,0], result[0:50,1], color='red', label='Setosa')
```

```
plt.scatter(result[50:100,0], result[50:100,1], color='green', label='Versicolor')
```

```
plt.scatter(result[100:150,0], result[100:150,1], color='blue', label='Virginica')
```

```
plt.title("Reduced Iris Dataset")
```

```
plt.xlabel("Dimension 1")
```

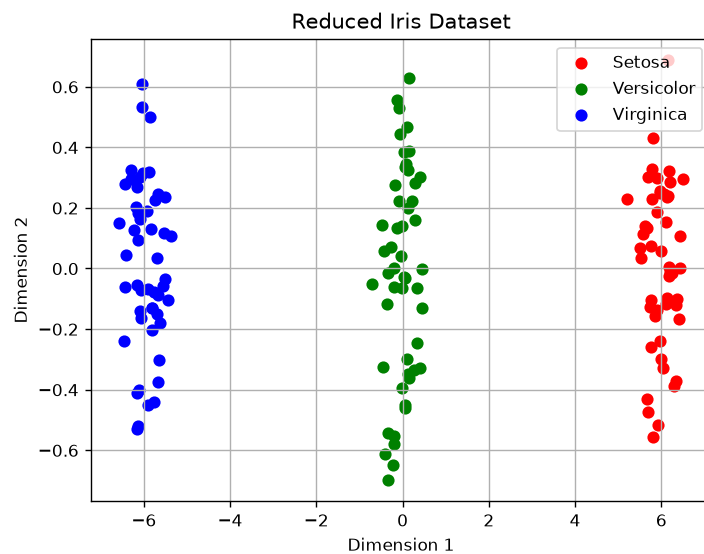
```
plt.ylabel("Dimension 2")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

Output:



15. An e-commerce company has collected customer purchasing behavior consisting of 60 attributes.

Reduce the dimensions using UMAP and identify customer clusters.

Python program:

```
import numpy as np
import matplotlib.pyplot as plt

# Create sample customer dataset
np.random.seed(40)
X = np.random.randint(1, 100, (150, 60))

# Mean centering
mean = np.mean(X, axis=0)
X = X - mean

# Covariance matrix
cov = np.cov(X.T)

# Eigenvalues and Eigenvectors
eigen_values, eigen_vectors = np.linalg.eigh(cov)
```

```

# Sort in descending order
index = np.argsort(eigen_values)[::-1]
eigen_vectors = eigen_vectors[:, index]

# Select first two principal components
pc = eigen_vectors[:, :2]

# Transform data
result = np.dot(X, pc)

# Plot customer clusters
plt.scatter(result[0:50,0], result[0:50,1], color='red', label='Cluster 1')
plt.scatter(result[50:100,0], result[50:100,1], color='green', label='Cluster 2')
plt.scatter(result[100:150,0], result[100:150,1], color='blue', label='Cluster 3')

plt.title("Customer Clusters")
plt.xlabel("Dimension 1")
plt.ylabel("Dimension 2")
plt.legend()
plt.grid(True)

plt.show()

```

Output:

